Object Oriented Programming (Lab) BSIT

- Fall 2024

Dr. Muhammad Farooq

The objective of this lab is to:

To understand and utilize the concept of classes (getters, setters) along with the previous concepts.

ALERT!

1. This is an individual lab. You are strictly NOT allowed to collaborate with others, share screens, or communicate answers in any form.

2. Use of AI tools (e.g., ChatGPT, Copilot, etc.) is strictly prohibited. Any AI-generated content will be treated as academic dishonesty.

3. Anyone caught in the act of cheating would be awarded a 0 in this Lab.

4. MAKE SURE TO VALIDATE WHERE EVER YOU TAKE INPUT FROM THE USER< OTHERWISE MARKS SHALL BE DEDUCTED.

LAB -06

Task 01: [10 Marks]

Implement a class named Person with the following definition.

```
class Person
{
        string firstName;
        string lastName;
        int age;
        char cnic[15];
        public:
```

```
        // TODO: Implement getter and setter methods for each data member

        // TODO: Implement this function to display all details

        void displayInfo();

        // TODO: Implement this function to check if the person is adult

        bool isAdult();

};
```

Your task is to:

1. **Write getter and setter methods** for all private data members.
2. **Implement the two functions** described below:

    - `displayInfo()` → prints all person details.
    - `isAdult()` → returns `true` if the person's age is 18 or above, otherwise `false`.

Input:

Enter First Name: Ali

Enter Last Name: Ahmad

Enter Age: 20

Enter CNIC: 042666888901

Output:

Name: Ali Ahmad

Age: 20

CNIC: 042666888901

The person is an adult.

Task 02: [10 Marks]

Implement a class named Time with the following definition.

```
class Time
{
   int hours;
   int minutes;
   int seconds;
public:
   // TODO: Implement getter and setter methods
   // TODO: Implement this function
   void setTime(int h, int m, int s);
   // TODO: Implement this function
   void displayTime();
   // TODO: Implement this function
   void addTime(Time t1, Time t2);
};
```

Your task is to:

1. **Write getter and setter methods** for all private data members (`hours`, `minutes`, `seconds`).
2. **Implement the following two functions:**
    - `void displayTime()` → prints the time in **HH:MM:SS** format.
    - `void addTime(Time t1, Time t2)` → adds two `Time` objects and stores the result in the current object.

Input:

Enter first time:

Hours: 2

Minutes: 45

Seconds: 50

Enter second time:

Hours: 1

Minutes: 30

Seconds: 30

Output:

Time 1: 2:45:50

Time 2: 1:30:30

Total Time: 4:16:20

Task 03: [10 Marks]

You are required to write a C++ program that dynamically allocates memory for a **1D array of integers**. After creating the array, you must **partition it into two sections** (left and right) **without creating any additional arrays**.
The partition must be performed **using pointer arithmetic only** — you will not use indexing (`[]`) or extra arrays to split the data.

Your program should:

1. Ask the user for the **total size** of the array.
2. Dynamically allocate memory for the array using the `new` operator.
3. Input the array elements from the user.
4. Ask for the sizes of the **left** and **right** partitions.
5. Ensure that the sum of the left and right partition sizes equals the total size of the array.
6. Use a **function** named

   int* partitionArray(int* arr, int leftSize, int rightSize, int totalSize);

to:

- Display the left partition using pointer arithmetic.
- Display the right partition using pointer arithmetic.
- Return a pointer to the **starting address of the right partition**.

Back in `main()`, display the right partition again using the pointer returned from the function.

Finally, deallocate the dynamically allocated memory using `delete[]`.

Input:

Enter total size of array: 5

Enter 5 elements:

10 20 30 40 50

Enter size of left partition: 2

Enter size of right partition: 3

Output:

Left Partition: 10 20

Right Partition: 30 40 50

Right Partition (from main): 30 40 50

Task 04: [10 Marks]

Write a C++ program that performs **advanced employee record management** using **file handling.**

The program should:

1. **Input Phase**
   - Ask the user to enter the number of employees `N`.
   - Dynamically allocate arrays to store:
     - Names
     - Ages
     - Salaries
     - Departments (string input)
2. **File Storage Phase**
   - Write all data into **two separate files**:
     - `personal.txt` → contains *Name* and *Age*
     - `official.txt` → contains *Name*, *Department*, and *Salary*
3. **Merging Phase**
   - Create a new file `merged.txt` that combines both files line by line (based on matching names).
     - Each line in `merged.txt` should look like this:
     - `Name: Ali | Age: 25 | Department: IT | Salary: 50000`

4. **Display Phase**
    - Read and display all merged data from `merged.txt` on the console.
5. **Processing Phase**
    - Calculate and display:
        - The **total number of employees in each department.**
        - The **average salary** of all employees.
        - The **employee with the highest salary.**
6. **Search Feature**
    - Allow the user to enter a name to **search** in the `merged.txt` file.
    - Display that employee's record if found, otherwise show *"Employee not found."*

EXAMPLE EXECUTION:

Enter number of employees: 3

Enter details for employee 1:

Name: Ali

Age: 25

Department: IT

Salary: 50000

Enter details for employee 2:

Name: Sara

Age: 30

Department: HR

Salary: 60000

Enter details for employee 3:

Name: Ahmed

Age: 28

Department: IT

Salary: 55000

Data successfully written to personal.txt and official.txt

Merging files into merged.txt...

Merged file created successfully!

---- Employee Records ----

Name: Ali | Age: 25 | Department: IT | Salary: 50000

Name: Sara | Age: 30 | Department: HR | Salary: 60000

Name: Ahmed | Age: 28 | Department: IT | Salary: 55000

---- Statistics ----

Total Employees in IT: 2

Total Employees in HR: 1

Average Salary: 55000

Highest Paid Employee: Sara (60000)

Enter name to search: Ahmed

Record found:

Name: Ahmed | Age: 28 | Department: IT | Salary: 55000